

Pygame Lesson 1 Notes

Summary of what we learned while building the Snake game.

1. What is Pygame?

Pygame is a Python library for making games. It provides tools for creating windows, drawing graphics, handling keyboard and mouse input, playing sounds, and running a game loop.

2. Basic Structure of a Pygame Program

1. import pygame
2. pygame.init()
3. Create the game window with `pygame.display.set_mode()`.
4. Start the game loop (while running:).
5. Process events.
6. Update game objects.
7. Draw everything.
8. Update the display.
9. Quit Pygame.

3. Important Functions

pygame.init() – Initializes Pygame.

pygame.display.set_mode() – Creates the game window.

pygame.display.set_caption() – Sets the window title.

pygame.event.get() – Retrieves all pending events.

pygame.draw.rect() – Draws a rectangle.

screen.fill() – Clears the screen with a color.

pygame.display.update() – Displays everything drawn this frame.

pygame.quit() – Closes Pygame.

4. Events

An event is something that happens, such as pressing a key, moving the mouse, or closing the window. `pygame.event.get()` returns a list of all events waiting to be processed.

5. Coordinates

Top-left corner = (0,0).

x increases to the right.

y increases downward.

6. Movement

dx = horizontal movement.

dy = vertical movement.

Right: dx = +20, dy = 0

Left: dx = -20, dy = 0

Up: dx = 0, dy = -20

Down: dx = 0, dy = +20

7. Food

We used `random.randrange(start, stop, step)`.

The step of 20 ensures the food appears on the same grid as the snake. The food does not move by itself. A new random position is generated only after it is eaten.

8. Equality Check

The snake eats the food when: `snake_x == food_x AND snake_y == food_y`

9. Why We Switched to a List

Instead of storing one position with `snake_x` and `snake_y`, we store every body segment: `snake = [[300,200], [280,200], [260,200]]` Each body segment is a list containing `[x, y]`.

10. Nested Lists

`snake[0]` -> first body segment.

`snake[0][0]` -> x-coordinate.

`snake[0][1]` -> y-coordinate.

11. `append()`

`append()` adds a new body segment to the end of the snake list. Example: `snake.append([280,200])`

12. Following Algorithm

Each body segment moves to the previous position of the segment in front of it. We discovered that moving from the head forward loses information, so the body must be updated from the tail backwards.

13. Key Ideas

- Think in coordinates.
- Lists can store many objects.
- Nested lists are common in games.
- The game loop repeats continuously.
- Every frame: process events, update positions, draw, update display.